

TEMA 093. TÉCNICAS DE DISEÑO DE SOFTWARE. DISEÑO POR CAPAS Y PATRONES DE DISEÑO.

Actualizado 19 Abril 2023

1. DISEÑO ESTRUCTURADO

Se ocupa de la **identificación, selección y organización de los módulos y sus relaciones**.

Objetivos: inteligibilidad, minimizar coste de mantenimiento, facilitar prueba, garantizar Reusabilidad, Fiabilidad y Portabilidad, utilización de herramientas gráficas para la representación modular

Principios generales DE

- ✓ Modularización (descomposición modular)
- ✓ Jerarquía entre módulos
- ✓ Independencia modular
- ✓ Modelización conceptual
- ✓ Principio de “caja negra” → abstracción. **Tipos Abstractos de Datos (TAD)** permiten generalizar y encapsular los aspectos relevantes sobre los datos que maneja un módulo o programa

Categorías de diseño:

- ✓ **Arquitectura general del sistema**
- ✓ **Diseño de alto nivel o arquitectónico o estructurado (modular): DEC - Estructura de Cuadros de Constantine.**
- ✓ **Diseño de bajo nivel o detallado:** Arquitectura de sistemas más comunes

1.1. EVALUACIÓN CALIDAD DEL DISEÑO

El **fan-out** de un módulo es usado como una **medida de complejidad**. Es el **número de subordinados inmediatos** de un módulo (cantidad de módulos invocados).

El **fan-in** de un módulo es usado como una **medida de reusabilidad**, es el **número de superiores inmediatos** de un módulo (la cantidad de módulos que lo invocan).

Descomposición: **Separar una función contenida en un módulo**, para que conforme un nuevo módulo.

Acoplamiento

Grado de interdependencia entre módulos. Se debe conseguir que los módulos sean lo más independientes entre sí.

Mínimo acoplamiento. Externo. Interfaz (parámetros intercambiados). De MENOR A MAYOR

- ✓ **Normal o simple:** los módulos sólo intercambian **datos**.
 - *De datos:* Los parámetros son **unidades elementales de datos**.
 - *Por estampado (de marca):* parámetros que pueden ser registros (**estructuras**).
 - *De control:* algún parámetro es de control, i.e., que controla las decisiones de los **módulos subordinados o superiores**.
- ✓ **Externo:** los módulos están **ligados a componentes externos** (controladores E/S, ...)
- ✓ **Común:** varios módulos hacen referencia a un **área común de datos** y pueden modificar los valores de los elementos de datos (p.e., variables globales).
- ✓ **De contenido:** Un módulo **hace referencia a la parte interna** de otro modificándola

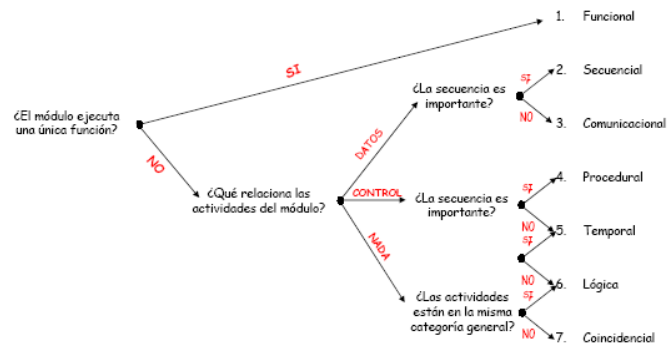
Cohesión

Definición: **Relación de los elementos de un módulo**. Alta cohesión: una tarea concreta y sencilla.

Máxima Cohesión. Características: **Interno**. De MAYOR A MENOR

- ✓ **Funcional:** Todos los elementos del módulo están relacionados; una única función.
- ✓ **Secuencial:** Un módulo empaqueta en secuencia varios módulos con cohesión funcional.
- ✓ **De comunicación:** Todos los elementos mismos datos de entrada y salida.
- ✓ **Procedimental o procedural:** Elementos relacionados y orden de ejecución. Paso de controles.
- ✓ **Temporal:** Tareas relacionadas que deben realizarse en el mismo intervalo de tiempo.

- ✓ **Lógica:** Sus elementos contribuyen en actividades de una misma categoría general.
- ✓ **Casual o coincidental:** Un módulo realiza un conjunto de tareas con poca o ninguna relación.



1.2. DIAGRAMAS DE ESTRUCTURA (O DIAGRAMA DE ESTRUCTURA DE CUADROS)

Objetivo: representar la **estructura modular del sistema** o de un **componente** y definir los parámetros.

Cada **nivel de la jerarquía** representa una descomposición más detallada del módulo del nivel superior.

Símbolos: **módulos** (entrada y salida, función, lógica interna y estado interno), **invocaciones** (relaciones entre módulos: secuencial, repetitiva, alternativa) y **cuplas** (comunicaciones entre módulos: control o flags, datos, modificada -flecha doble-, de resultados).

Almacén de datos: Representación física del lugar donde están almacenados los datos.

Dispositivo físico: dispositivo por el cual se puede recibir o enviar información que necesite el sistema.

1.3. ESTRATEGIAS DE DISEÑO ESTRUCTURADO

Permite una **transición del DFD a una descripción de diseño de la estructura del programa**. Estrategias:

- Flujo de transformación:** flujo de entrada, transformación, flujo de salida.
- Flujo de transacción:** flujo de **datos de entrada**, **evaluación de la transacción**, **selección del camino de acción** a ejecutar en función de la evaluación.

2. DISEÑO ORIENTADO A OBJETOS

El diseño orientado a objetos se define según Booch como un “*método de diseño abarcando el proceso de descomposición orientado a objetos y una notación para representar ambos modelos lógico y físico tal como los modelos estáticos y dinámicos del sistema bajo diseño*”.

2.1. CARACTERÍSTICAS

- ✓ **Modularidad**
- ✓ **Ocultación de implementación (information hiding)**
- ✓ **Acoplamiento débil**
- ✓ **Cohesión fuerte**
- ✓ **Extensibilidad.** Se permite gracias a **herencia** y **polimorfismo**
- ✓ **Integrable**
- ✓ **Reusabilidad**

2.2. PRINCIPIOS SOLID

SOLID. Principios que favorecen el diseño y la mantenibilidad del software orientado a objetos:

- ✓ **S – The Single Responsibility Principle (SRP):** Principio de Responsabilidad Única
“Una clase debería tener solo una razón para cambiar”
- ✓ **O – The Open/Closed Principle (OCP):** Principio Abierto / Cerrado
“Una clase debería poder ser extendida, sin ser modificada”
- ✓ **L – The Liskov Substitution Principle (LSP):** Principio de Sustitución de Liskov
“Clases derivadas deben ser sustituibles por sus clases bases”
- ✓ **I – Interface Segregation Principle (ISP):** Principio de Segregación de Interfaces
“Muchas interfaces muy especializadas son preferibles a una interfaz general en la que se agrupen todas las interfaces”
- ✓ **D – The Dependency Inversion Principle (DIP):** Principio de Inversión de Dependencias
“Las abstracciones no deben depender de los detalles, los detalles deben depender de las abstracciones”

2.3. PRINCIPIOS GRASP

GRASP (object-oriented design General Responsibility Assignment Software Patterns) o conjunto de buenas prácticas en el diseño orientado a objetos, según lo describe Craig Larman:

- ✓ Experto en Información.
- ✓ Creador.
- ✓ Alta Cohesión.
- ✓ Bajo Acoplamiento.
- ✓ Polimorfismo.
- ✓ Fabricación pura.
- ✓ Indirección.
- ✓ Variaciones protegidas.
- ✓ Controlador.

3. PROGRAMACIÓN ORIENTADA A ASPECTOS

Paradigma de programación reciente que busca modularización y separación de responsabilidades. Ventajas: código menos enmarañado, menor dependencia, facilidad para depurar y modificar, reusabilidad.

Fundamentos: lenguaje para funcionalidad básica, lenguaje de aspectos (AspectJ) y tejedor.

Conceptos básicos:

- Aspecto (Aspect)
- Punto de cruce o unión (Join point)
- Consejo (Advice)
- Puntos de corte (Pointcut)
- Introducción (Introduction)
- Destinatario (Target)
- Resultante (Proxy)
- Tejido (Weaving)

4. DISEÑO POR CAPAS

Consigue separación e independencia. La funcionalidad del sistema se organiza en capas y cada una se apoya en las facilidades y servicios ofrecidos por la capa bajo ella. Desarrollo incremental y portabilidad.

Dos capas: Separación entre capa de presentación (usuario/cliente) y capa de aplicación y datos.

Middleware: Nivel de indirección entre los clientes y las demás capas del sistema.

Tres capas o multicapa:

- Presentación. Usuario/cliente
- Aplicación. Lógica de negocio
- Capa de gestión de datos. Modelo de datos

Cliente/servidor. Modelo de aplicación distribuida en el que las tareas se reparten entre los proveedores de recursos o servicios (servidores) y los demandantes de servicios o clientes. Un ejemplo es la web.

- Cliente pesado (fat client)
- Cliente ligero (thin client)

5. PATRONES DE DISEÑO

Es una solución “repetible” a un problema “común” de diseño. Comprobada efectividad y ser reutilizable.

Orígenes: 1979 por el arquitecto Christopher Alexander.

Software: años 90. “Design Patterns” escrito por el grupo Gang of Four (GoF).

CATEGORÍAS

- ✓ Patrones de arquitectura. Esquema organizativo estructural para sistemas software.
- ✓ Patrones de diseño. Esquemas para definir estructuras de diseño software.
- ✓ Dialectos. Bajo nivel, para un lenguaje de programación o entorno.

5.1. PATRONES GOF

5.1.1 PATRONES CREACIONALES

- ✓ **Object pool** (conjunto de objetos). Nuevos objetos a través de clonación.
- ✓ **Abstract Factory** (Factoría abstracta). Crear objetos de una misma familia
- ✓ **Builder** (Constructor virtual). Permitir la creación de una variedad de objetos complejos
- ✓ **Factory Method** (Método de Fabricación). Uso de clase constructora.
- ✓ **Prototype** (Prototipo). Crea nuevos objetos clonándolos de una instancia ya existente
- ✓ **Singleton** (Instancia única). Única instancia de una clase y mecanismo global de acceso.

5.1.2 PATRONES ESTRUCTURALES

- ✓ **Adapter** (Adaptador). Adapta una interfaz en otra
- ✓ **Active record (Registro activo):** en el ORM para que el objeto gestione su propia persistencia en una BD relacional
- ✓ **Bridge** (Puente): Desacopla una abstracción de su implementación
- ✓ **Composite** (Objeto compuesto): Permite tratar objetos compuestos como uno simple
- ✓ **Decorator** (Envoltorio): Añade funcionalidad a una clase dinámicamente
- ✓ **Facade** (Fachada): Provee de una interfaz unificada simple para acceder a una interfaz o grupo
- ✓ **Flyweight** (Peso ligero): Reduce la redundancia cuando gran cantidad de objetos
- ✓ **Proxy:** Proporciona un intermediario o representante de un objeto para controlar su acceso.

5.1.3 PATRONES DE COMPORTAMIENTO

- ✓ **Chain of Responsibility (Cadena de responsabilidad):** Permite establecer la línea que deben llevar los mensajes para que los objetos realicen la tarea indicada.
- ✓ **Command (Orden):** Encapsula una operación en un objeto.
- ✓ **Interpreter (Intérprete):** Dado un lenguaje, define una representación para su gramática.
- ✓ **Iterator (Iterador):** Permite realizar recorridos sobre objetos compuestos.
- ✓ **Mediator (Mediador):** Define un objeto que coordine la comunicación entre objetos
- ✓ **Memento (Recuerdo):** Almacenar el estado de un objeto en un momento dado
- ✓ **Observer (Observador):** Define una dependencia de uno-a-muchos entre objetos; notificación y actualización
- ✓ **State (Estado):** Modificar comportamiento de un objeto cuando cambie su estado interno.
- ✓ **Strategy (Estrategia):** Permite disponer de varios métodos para resolver un problema y elegir.
- ✓ **Template Method (Método plantilla):** Define en una operación el esqueleto de un algoritmo.
- ✓ **Visitor (Visitante):** Permite separar el algoritmo de la estructura de un objeto.

5.2. PATRONES DE DISEÑO MODERNOS

- ✓ **Dependency Injection:** Técnica para crear objetos que dependen de otros objetos.
- ✓ **Event Sourcing:** Se utiliza en sistemas que requieren la persistencia de eventos. En lugar de almacenar el estado actual del sistema, se almacenan todos los eventos que han ocurrido.
- ✓ **CQRS (Command Query Responsibility Segregation):** Separa las operaciones de lectura y escritura en dos modelos diferentes, para ayudar a mejorar el rendimiento y la escalabilidad del sistema.
- ✓ **Patrón Saga:** Se utiliza en sistemas distribuidos y es especialmente útil para el manejo de transacciones a largo plazo. Se utiliza para garantizar que todas las operaciones dentro de una transacción se completen correctamente o se anulen en caso de error.